

AI Hiring Intelligence Platform

Complete Interview & Explanation Guide — Explainable Agentic RAG resume screening

PART 1 — The Complete Explanation

1. The 30-Second Pitch

"I built an explainable AI resume-screening platform. Unlike a keyword ATS, it uses an Agentic RAG pipeline — two AI agents where one gathers evidence from the resume using semantic search, and the other reasons over that evidence to produce a hiring report. Every score is traceable back to a specific line in the resume, so a recruiter sees WHY a candidate matched, not just a number. It runs fully offline with a deterministic engine, and plugs into an LLM like Gemini for richer reasoning."

Differentiators: explainable · evidence-first · reasons about transferable skills (not just keywords) · works with or without an LLM.

2. Problem Statement

The problem: Companies get hundreds of resumes per job. Traditional ATS (Applicant Tracking Systems) filter by keyword matching — if the JD says "PostgreSQL" and your resume says "SQL", you get rejected even though they're the same skill. Two failures:

- **False rejections** — good candidates dropped because their wording differs (a Django dev rejected for a "FastAPI" role, even though it's a trivial jump).
- **No explanation** — recruiters get a score with no reasoning, so they can't trust or defend it.

My solution: an evaluator that (a) understands skills semantically (SQL ≈ MySQL), (b) reasons about transferable and implied skills, and (c) makes every decision explainable: Requirement → Evidence → Reasoning → Confidence → Decision.

Why it matters: hiring decisions are high-stakes and increasingly legally scrutinised ("why was this candidate rejected?"). An explainable, auditable system is the real value.

3. Tech Stack (and WHY each)

Layer	Tech	Why this choice
Backend framework	FastAPI (Python)	Async, auto OpenAPI docs, Pydantic validation, fast.
Orchestration	LangGraph	Models the agent pipeline as a state graph — explicit, debuggable, no hidden control flow.
Embeddings	BAAI/bge-small-en-v1.5 via FastEmbed (ONNX)	Small (384-dim), strong retrieval, CPU-local. ONNX = no PyTorch = light memory.
Vector search	FAISS (IndexFlatIP)	Fast exact similarity search, runs locally.

Keyword search	Custom BM25	Sparse retrieval to complement semantic search (hybrid).
LLM (optional)	OpenAI / Anthropic / Gemini via LlamaIndex	Config-driven, no vendor lock-in; deterministic fallback if no key.
Database	SQLite + SQLAlchemy	Zero-config, file-based, self-contained.
Doc parsing	PyMuPDF (PDF), python-docx (DOCX)	Robust text + page extraction.
Frontend	React 19 + TypeScript + Vite + Tailwind	Fast dev, typed API contract, modern UI.
Orchestration glue	LlamaIndex	Connects embeddings → vector store → retrieval → LLM.

Testing: pytest (143 tests), oxlint, tsc.

4. Core Concept: Agentic RAG

RAG = Retrieval-Augmented Generation. Instead of asking an LLM from memory (where it hallucinates), you first retrieve relevant facts, then reason **ONLY** over those facts. It grounds output in real evidence. **Agentic** = the retrieval and reasoning are orchestrated by autonomous agents.

The two-agent design (why exactly two):

```
Resume + JD
|
v
[ Agent 1: Candidate Intelligence Agent ]    <- the INVESTIGATOR
  parse -> chunk -> embed -> FAISS -> retrieve evidence
    (gathers evidence, never judges)
    |   (structured evidence package)
    v
[ Agent 2: Hiring Decision Agent ]           <- the DECISION-MAKER
  score -> explain -> recommend -> questions
    (reasons over evidence ONLY, never sees the raw PDF)
    |
    v
Explainable Hiring Report
```

Why two agents? It mirrors real hiring: one party collects evidence, another decides. Separating "collect" from "decide" makes every conclusion auditable — Agent 2 can only use what Agent 1 found, so there is no black box. Agent 2 never touches the raw document, which structurally prevents hallucination.

Implemented as a LangGraph state machine: START → candidate_intelligence → hiring_decision → END. Sequential, no branching.

5. The Algorithms

5a. Chunking — section-aware + sentence-aware

Detect sections (Experience/Projects/Skills) by header patterns, then split each section on sentence boundaries (~350 chars, max 600, 1-sentence overlap). Section labels let us weight Experience evidence higher than a Skills-list mention.

5b. Embeddings & cosine similarity

An embedding turns text into a vector where similar meaning = nearby vectors. Similarity = cosine (angle between vectors: 1 = identical, 0 = unrelated). Because BGE vectors are L2-normalised, cosine = dot product = exactly what FAISS IndexFlatIP computes. Example: cosine("SQL", "MySQL") $\approx$ 0.84 — the model KNOWS they're related with no hardcoded rule.

5c. Retrieval — hybrid: dense + sparse + fusion

- **Dense (semantic):** embed the requirement, find nearest chunks in FAISS.
- **Sparse (BM25):** classic keyword scoring that rewards exact term matches.
- **Fusion (Reciprocal Rank Fusion):** combine both rankings: $\text{score} = \sum \text{weight} / (k + \text{rank})$.
- **Similarity floor (0.62):** a chunk must clear a cosine threshold to count, so an absent skill returns nothing → correctly "Missing".

5d. Requirement extraction — generic, not a keyword list

Two layers: (1) known-tech taxonomy, (2) grammar cues ("experience with X", "5+ years", capitalized phrases). Generalises to any technology (Rust, SAP, Unity) without new rules.

5e. Prioritization

Reads JD wording (headers/cues) to tag each requirement must (weight 1.0) or nice (0.4). Coverage is importance-weighted, so missing a must-have hurts more.

5f. Authenticity / keyword-stuffing detection

A skill is "demonstrated" only if it appears in a narrative section (Experience/Projects). A skill only in a Skills list = over-claimed (keyword stuffing). Produces a credibility score and risk level. Fully deterministic.

5g. Candidate profile & seniority

Regex/heuristics extract name, title, years, seniority band (Junior→Lead), compared vs the JD's "N+ years" → Below/Meets/Exceeds. No hardcoded names.

5h. Scoring — deterministic, reproducible

```
overall = coverage x0.35 + experience x0.25 + project x0.20 + confidence x0.15 + quality x0.05
```

Each sub-score 0-100, computed by formula (not the LLM), so the same input always gives the same number — fair and defensible.

5i. Transfer & equivalence reasoning (the "smart" part — generic, no hardcoded maps)

- ≥ 0.80 → **Equivalent:** practically the same skill (SQL satisfies MySQL). Rescues a "Missing" requirement.
- ≥ 0.68 + **same category** → **Related:** adjacent skill; keeps status honest but notes "quick ramp-up from your X experience".
- **Below** → **genuine gap.**

Key insight: it's algorithms over examples — thresholds are calibrated, but the relationships come from the embedding model's knowledge, so it works for skills never seen before.

5j. LLM reasoning (when a key is configured)

The LLM (Gemini) gets ONLY the retrieved evidence + a prompt to reason about equivalence, implied skills, transferability, and depth in plain language, and to generate transfer-aware interview questions. It never sees the raw resume. The LLM owns judgment; algorithms own the numbers.

5k. AI Recruiter (self-interview)

For each interview question, the AI answers it as the candidate would, using only their resume evidence, then gives: ideal answer, supporting evidence, confidence, missing info, follow-up probes, and a recruiter verdict.

6. THE Design Principle: Deterministic vs LLM

"I deliberately split the work: deterministic algorithms own everything that must be reproducible — embeddings, similarity, scoring, coverage, authenticity. The LLM only does what genuinely needs language reasoning — explanations, recommendations, interview questions. And crucially, the whole thing works with no LLM at all via a deterministic fallback engine."

Why: reproducibility (scores can't drift run-to-run), no vendor lock-in / cost control (works offline), no hallucinated scores (the LLM can't invent a number; it only writes prose).

7. Real Challenges I Solved

- 1. Stale-index contamination bug.** A frontend developer's resume "matched" every backend requirement. The FAISS index was keyed only by resume_id — a DB reset reused IDs and loaded the wrong resume's embeddings. Fix: content-address the index (hash the file, store the fingerprint, rebuild on mismatch).
- 2. Similarity-floor calibration.** At a low threshold (0.30) a React resume scored 74/100 on a backend job. I measured actual cosine separation (genuine ~0.62+, spurious ~0.56-0.61) and raised the floor to 0.62 — after which an off-domain resume correctly scored 0. Data-driven.
- 3. Precision/recall tension.** A single cosine threshold that accepts Docker→Kubernetes also wrongly accepts Azure→SQL. Fix: require the "related" claim to share the skill's category. And I stayed honest: the small embedding model can't know Django→FastAPI by name — which is exactly why the LLM path exists.
- 4. Deploy memory footprint.** PyTorch made the image too heavy for 512MB free hosts. Swapped embeddings to ONNX (FastEmbed) — same weights, identical results, ~4x less RAM.

8. Hosting & Deployment — the concept

What is hosting? Running your app on an always-on cloud server with a public URL so anyone can use it (the client-server model: the browser is the client, your server responds over HTTP).

Why Docker / containerization? A container packages your app WITH its exact dependencies into one portable image — it solves "works on my machine" by running identically everywhere. Like a standardized shipping container.

How MY app deploys: one container builds the React frontend and has FastAPI serve it at the same

Origin (one URL, no CORS). The Dockerfile is multi-stage (Node builds frontend, Python runs backend + serves it, port 7860). Config comes from environment variables; the API key is never baked into the image or committed — it's a runtime secret.

Free-tier constraint: free tiers give ~512MB RAM. I made it fit by removing PyTorch (ONNX instead). Targets that take a Dockerfile: Render (with a render.yaml blueprint), Koyeb, or Hugging Face Spaces. Steps: push to GitHub → create host account → connect repo → it builds the Dockerfile → public URL → add the LLM key as a secret env var.

9. Likely Interview Questions + Strong Answers

Q: What is RAG and why use it here?

Retrieval-Augmented Generation — retrieve relevant facts first, then reason over them, instead of trusting model memory. Here it grounds every hiring conclusion in real resume evidence → explainability, no hallucination.

Q: Why two agents instead of one prompt?

Separation of concerns — one gathers evidence, one decides. Makes the pipeline auditable and mirrors real hiring; the decider never sees the raw doc, which prevents hallucination.

Q: How does semantic search work?

Text → embedding vectors; similar meaning = nearby vectors; measured by cosine. Vectors normalised so cosine = dot product, computed by FAISS inner-product index.

Q: Why hybrid retrieval?

Dense embeddings capture meaning but blur exact terms; BM25 captures exact keywords but not meaning. Fusing them (RRF) gives both.

Q: How do you avoid hardcoding skill relationships?

No manual maps. Cosine similarity with calibrated thresholds (≥ 0.80 equivalent, ≥ 0.68 + same category related). Relationships come from the model's learned knowledge, so it generalises to any tech.

Q: What if there's no LLM / it fails?

A deterministic engine produces the entire report; the LLM is an enhancement, never a hard dependency, and it falls back gracefully on any error.

Q: How would you scale it?

FAISS IndexFlatIP is exact/linear — for many candidates move to approximate indexes (IVF/HNSW) or a vector DB (Qdrant/pgvector), batch the embeddings, parallelize ranking.

Q: Biggest limitation?

The small embedding model can't capture deep domain relationships by name; a single similarity floor trades precision vs recall — the deliberate reason the LLM path exists.

10. One-line summaries to memorize

- **Architecture:** two-agent Agentic RAG on a LangGraph state machine.
- **Principle:** algorithms own the numbers, the LLM owns the language, works offline.

- **Retrieval:** hybrid — semantic (BGE+FAISS cosine) + keyword (BM25) fused by RRF.
- **Explainability:** Requirement → Evidence → Reasoning → Confidence → Decision.
- **Reasoning:** generic transfer/equivalence via embedding cosine — no hardcoded skill maps.
- **Deploy:** single Docker container, ONNX embeddings (no torch) so it fits free tiers.

PART 2 — End-to-End Simulation

One resume traced through every phase, with the tools used at each step.

The two documents we start with

Resume (priya_sharma.pdf):

```
Priya Sharma
Backend Developer
Experience
Built REST APIs for an e-commerce platform using Django and Python.
Designed SQL database schemas and optimized queries handling 2M+ rows.
Deployed services with Docker on AWS.
Projects
Built an image-classification model using PyTorch for a college project.
Skills
Python, Django, SQL, Docker, AWS, PyTorch, Git, REST API
```

Job Description (senior_backend.pdf):

```
Senior Python Backend Engineer
Requirements:
- Strong Python and FastAPI
- MySQL database experience
- 4+ years backend experience
Nice to have:
- TensorFlow
- Kubernetes
```

Keep this in mind: A traditional ATS scans for the exact words "FastAPI", "MySQL", "TensorFlow", "Kubernetes" — Priya's resume has NONE of them. An ATS rejects her instantly. Watch what our system does instead.

PHASE 0 — Upload

Recruiter drags the PDF in. Saved to disk as 1_priya_sharma.pdf; a DB row is created.

Tools: React (UI) → Axios (HTTP) → FastAPI (POST /api/v1/resume/upload) → SQLAlchemy → SQLite.

PHASE 1 — Document Parsing

The PDF text is extracted, with page markers injected.

Tools: PyMuPDF (fitz) for PDF, python-docx for Word.

PHASE 2 — Semantic Chunking

Split into section-aware, sentence-aware chunks, each a LlamaIndex TextNode with metadata (section,

page).

Where LlamaIndex is used (1st time): chunks are packaged as LlamaIndex TextNode objects — the standard unit for indexing/retrieval.

```
Chunk 0 [Experience, p1]: "Built REST APIs ... using Django and Python."
Chunk 1 [Experience, p1]: "Designed SQL database schemas ... Docker on AWS."
Chunk 2 [Projects, p1]: "Built an image-classification model using PyTorch..."
Chunk 3 [Skills, p1]: "Python, Django, SQL, Docker, AWS, PyTorch, Git, REST API"
```

PHASE 3 — Vector Embedding

Each chunk becomes a 384-dimensional vector capturing its MEANING (similar meaning → nearby vectors).

Tools: FastEmbed (ONNX) running BAAI/bge-small-en-v1.5, wrapped by LlamaIndex FastEmbedEmbedding.

Where embedding happens: `get_embedding_model().get_text_embedding(chunk)` produces the vector.

```
"Designed SQL database schemas..." ->
[ 0.041, -0.118, 0.073, 0.202, -0.055, 0.161, ... 378 more ... ]
```

This is what makes it semantic: the vector for "SQL" sits close to "MySQL" even though the words differ.

PHASE 4 — Building the FAISS Vector Index

All 4 chunk-vectors are stored in a FAISS index (built for fast similarity search), saved per-resume to disk.

Tools: FAISS (IndexFlatIP) via LlamaIndex VectorStoreIndex + FaissVectorStore.

Fingerprint trick: we store a hash of the resume next to the index; if the file changes, we rebuild — so we never evaluate the wrong resume.

Why IndexFlatIP? Our vectors are unit-normalised, so inner product = cosine similarity, computed in microseconds.

PHASE 5 — JD Parsing + Requirement Extraction

Extract the requirement list — not from a fixed keyword list, but via taxonomy + grammar cues. Works for any technology.

```
["python", "fastapi", "mysql", "4+ years experience", "tensorflow", "kubernetes"]
```

PHASE 6 — Prioritization (must vs nice)

```
python -> must (1.0)    fastapi -> must (1.0)    mysql -> must (1.0)
4+ years -> must (1.0) tensorflow -> nice (0.4) kubernetes -> nice (0.4)
```

PHASE 7 — Retrieval (the core of RAG)

For each requirement we hybrid-search Priya's index. Two traces:

Requirement: "python"

- Embed "python" → query vector (LlamaIndex FastEmbedEmbedding again — embedding at query time).
- Dense (FAISS): Chunk 0 "...Django and Python" → cosine 0.71; Chunk 3 "Python, Django, SQL..." → 0.66.
- Sparse (BM25): "python" literally present → high score. Fusion (RRF): Chunk 0 wins.
- Floor 0.62: both pass. Verdict: MATCHED (Experience, high confidence).

Tools: LlamaIndex retriever (dense) + custom BM25 (sparse) + Reciprocal Rank Fusion.

Requirement: "mysql" — the money shot

- Embed "mysql". Dense: nearest is Chunk 1 ("SQL database schemas") → cosine 0.60.
- Floor 0.62: $0.60 < 0.62$ → DROPPED. By pure retrieval, "mysql" is Missing (an ATS also fails here).
- Then equivalence reasoning: candidate skills = {python, django, sql, docker, aws, pytorch, rest api}. $\text{cosine}(\text{"mysql"}, \text{"sql"}) = 0.84 \geq 0.80$ → EQUIVALENT.
- Verdict: MATCHED — "You list SQL, which is practically the same skill as MySQL."

THIS is the power over an ATS. The ATS sees "no MySQL" → reject. We understand SQL IS MySQL for practical purposes, using the model's knowledge, with zero hardcoded rules.

PHASE 8 — Deterministic signals

Profile: name "Priya Sharma", title "Backend Developer", years vs JD's 4+ → seniority fit. **Authenticity:** Python/SQL/Docker are in Experience (genuine), not just listed → high credibility, low stuffing risk.

PHASE 9 — The Evidence Package (Agent 1's output)

Everything Agent 1 gathered, bundled into one JSON — the only thing Agent 2 sees (never the raw PDF).

```
{ "candidate_profile": {...}, "authenticity": {...},
  "retrieval_results": [
    { "requirement": "python", "importance": "must", "matches": [{"chunk": "...Django and Python", "section": "Experience", "score": 0.71}] },
    { "requirement": "mysql", "importance": "must", "matches": [], ... } ] }
```

PHASE 10 — Reasoning (Agent 2 decides)

Deterministic engine (no LLM): python → Matched; mysql → Matched via equivalence (0.84); fastapi → Missing (Django~FastAPI 0.58 below threshold for the small model); tensorflow → Missing but transferable ("PyTorch experience is closely related — quick ramp-up").

LLM engine (Gemini): gets the same evidence + a prompt to reason about equivalence/transfer in plain language. The LLM KNOWS Django→FastAPI are cousins (small embeddings don't) → "Strong Django + REST API experience — FastAPI is a fast transition."

Tools: evaluation_service (deterministic) OR LlamaIndex LLM (GoogleGenAI for Gemini).

Where LlamaIndex is used (again): the LLM is called through LlamaIndex's provider-agnostic interface — swap OpenAI/Anthropic/Gemini with one config line.

PHASE 11 — Scoring


```
coverage 72, experience 65, project 40, confidence 70, quality 88
overall = 72x.35 + 65x.25 + 40x.20 + 70x.15 + 88x.05
         = 25.2 + 16.3 + 8.0 + 10.5 + 4.4 ~ 64 -> "Conditional Match"
```

Priya scores ~64/100 — a real interview candidate. The ATS scored her 0.

PHASE 12 — The Report

A full HiringReport: scores, per-requirement verdicts with evidence, strengths/weaknesses, skill gaps + learning roadmap, transfer-aware interview questions ("You've used PyTorch — how would you approach the same in TensorFlow?"), recommendation, authenticity block. Saved to SQLite + a JSON file.

PHASE 13 — Frontend + Recruiter Tools

Tools: React + TypeScript + Tailwind, inline SVG charts.

The recruiter can rank Priya against others, view analytics, move her through the workflow, add notes, export a PDF, and run the AI Recruiter (which answers each interview question from her resume so the recruiter can judge if she can back up her skills).

How powerful is our RAG vs a traditional ATS?

	Traditional ATS	Our Agentic RAG
Matching	Exact keyword match	Semantic (meaning-based)
SQL vs MySQL	Different → reject	Recognised equivalent (0.84)
Django dev for FastAPI role	Reject	Transferable / LLM sees the link
PyTorch for TensorFlow	Reject	"Adjacent — quick ramp-up"
Priya's outcome	Score 0 → auto-rejected	Score 64 → interview
Explanation	None (just a %)	Requirement→Evidence→Reasoning→Confidence
Keyword stuffing	Gets fooled	Detected (must be in Experience)
New/unknown tech	Needs manual rules	Generalises automatically
Interview help	None	AI Recruiter simulates the interview

The one-liner for HR: "A keyword ATS would have auto-rejected Priya because her resume says 'SQL' and 'Django', not 'MySQL' and 'FastAPI'. Our system understands that SQL IS MySQL and that a Django developer picks up FastAPI easily — so it scores her a 64 and sends her to interview, with a full explanation of why. It reads a resume the way an experienced recruiter does, not like a find-and-match tool — and it shows its reasoning so you can trust it."

Where each tool lives (quick reference)

- **LlamaIndex** — the backbone: TextNode (chunks), FastEmbedEmbedding (embeddings), VectorStoreIndex+FaissVectorStore (index), retriever (search), provider-agnostic LLM (reasoning).
- **Vector embedding** — FastEmbed/ONNX running BGE-small (Phase 3 = chunks, Phase 7 = queries).

- • **FAISS** — the vector index + cosine search.
- **BM25** — keyword search, fused with FAISS via RRF.
- **PyMuPDF** / **python-docx** — document parsing.
- **FastAPI** + **SQLAlchemy** + **SQLite** — API + storage.
- **Gemini (via LlamaIndex)** — optional LLM reasoning; deterministic engine is the fallback.